

Offer Letter:

CIN: U92490GJ2022PTC136763

**Internship Confirmation Letter**

Date: 24-Apr-2026

Intern ID: AG642**To: Mr. Prince Akbari**

We are pleased to confirm your selection for the internship program at AGEVOLE INNOVATION PVT. LTD. You have been selected as an AI/ML Developer. The tenure of internship with AGEVOLE INNOVATION PVT. LTD. is 1.5 Months and will start from May 13th, 2026 and will end on Jun 21st, 2026.

We are happy to have you work with us.

Yours Sincerely,



Masum Patel
Co-founder & CTO
AGEVOLE INNOVATION PVT. LTD.

Note: Verify Document Here by Intern ID on <https://verify.agevole.in/>.

Internship Certificate:

CIN: U92490GJ2022PTC136763



Certificate of Internship

Date: 22-Jun-2026

Intern ID: AG642

To: Mr. Prince Akbari

This is to certify that Mr. Prince Akbari has successfully completed 1.5 months of internship as a AI/ML Developer at AGEVOLE INNOVATION PVT. LTD., from May 13th, 2026 to Jun 21st, 2026. During this period, He consistently demonstrated excellent technical skills, a proactive approach to problem-solving, and a professional attitude. His contributions added significant value to our development team and projects.

We are glad that you have worked with us.

Yours Sincerely,



Masum Patel
Co-founder & CTO
AGEVOLE INNOVATION PVT. LTD.

Note: Verify Certificate Here by Intern ID on <https://verify.agevole.in/>.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Agevole Innovation Pvt. Ltd., Surat, for granting me the opportunity to undertake my summer internship and to work on a practical, industry-relevant machine learning project.

I am deeply thankful to my external guide at Agevole Innovation for their continuous support, technical mentorship, and valuable feedback throughout the Development of the BigMart Sales Prediction System, which helped me understand real-world data science workflows beyond the classroom.

I would also like to thank my internal guide and the Chandubhai S. Patel Institute of Technology, for their guidance, encouragement, and for providing the academic framework that made this internship possible.

I am also grateful to the technical and non-teaching staff of the department for their timely assistance with laboratory resources and software installations required during Development and testing.

Finally, I extend my heartfelt thanks to my family and friends for their constant support and motivation during the course of this internship.

Prince Akbari
24CS003

ABSTRACT

Retail chains generate vast amounts of transactional data every day, and accurately forecasting product-level sales at individual outlets is essential for effective inventory planning, pricing strategy, and outlet management. This report presents the work carried out during a six-week summer internship at Agevole Innovation Pvt. Ltd., Surat, on the design and Development of the BigMart Sales Prediction System — an end-to-end machine learning pipeline and interactive web application that forecasts item-level sales across BigMart retail outlets.

The project uses the publicly available BigMart Sales dataset, consisting of approximately 8,500 historical records described by 12 attributes covering item and outlet characteristics. The scope of the internship included data cleaning and feature engineering (handling inconsistent categorical labels, imputing missing item visibility values, deriving outlet age and item category features, and MRP-based binning), followed by training and comparing four regression models — Linear Regression, Decision Tree, Random Forest, and XGBoost — using a Scikit-Learn preprocessing and modelling pipeline. The best-performing model, selected using the R^2 score on a held-out test split, was serialized and exposed through a Flask REST API, and consumed by an interactive Streamlit dashboard offering prediction, scenario comparison, analytics, and model-diagnostic views, with visualizations built using Plotly.

The resulting system demonstrates a complete, deployable machine learning workflow — from raw data to a usable prediction interface — and provides BigMart with a practical tool to estimate expected sales for a given item–outlet combination, compare hypothetical scenarios, and explore the historical drivers of sales performance. The report also documents the system design, module-level implementation, and testing carried out during the internship, and concludes with the limitations of the current system and recommendations for future enhancement, such as real-time data integration and more advanced model tuning.

Keywords: Retail Sales Forecasting, Machine Learning, Regression, XGBoost, Random Forest, Flask REST API, Streamlit Dashboard, Feature Engineering, Data Analytics.

TABLE OF CONTENTS

Certificate

Company / Organization Certificate

Acknowledgementi

Abstractii

List of Figuresv

List of Tablesvi

List of Abbreviationsvii

Description of Company / Organizationviii

CHAPTER 1: PROJECT DEFINITION1

1.1 Introduction1

1.2 Problem Statement1

1.3 Objectives1

1.4 Scope2

1.5 Methodology Overview2

1.6 Internship Timeline / Work Breakdown2

CHAPTER 2: LITERATURE REVIEW4

2.1 Traditional Approaches to Sales Forecasting4

2.2 Machine Learning Based Forecasting4

2.3 Related Work on the BigMart Dataset4

2.4 Research Gap Addressed by this Project4

2.5 Overview of Regression Algorithms Used5

CHAPTER 3: SYSTEM ANALYSIS AND DATASET DESCRIPTION7

3.1 Dataset Description7

3.2 Exploratory Data Analysis7

3.3 Data Cleaning & Feature Engineering8

3.4 Modelling Approach9

CHAPTER 4: SOFTWARE AND HARDWARE REQUIREMENTS10

4.1 Software Requirements10

4.2 Hardware Requirements10

4.3 Development Environment Notes10

CHAPTER 5: SYSTEM DESIGN12

5.1 System Architecture12

5.2 Use Case Diagram12

5.3 Data Flow Diagram13

5.4 Database / Model Artifact Design14

5.5 Deployment View	14
CHAPTER 6: MAJOR FUNCTIONALITY AND IMPLEMENTATION	15
6.1 Model Training Pipeline (train_model.py)	15
6.2 Flask REST API (predict.py)	16
6.3 Streamlit Dashboard (app.py)	17
6.4 Key Implementation Snippets	17
6.5 Running the Project Locally	18
CHAPTER 7: FLOW CHART	19
CHAPTER 8: TESTING	20
8.1 Testing Strategy	20
8.2 Unit and Integration Test Cases	20
8.3 API Testing	20
8.4 Non-Functional and Usability Testing	21
CHAPTER 9: SCREENSHOTS OF PROJECT OUTPUT	22
9.1 Predict Page	22
9.2 Compare Page	23
9.3 Analytics Page	23
9.4 Model Diagnostics	24
CHAPTER 10: RESULTS AND DISCUSSION	25
CHAPTER 11: LIMITATIONS OF PROJECT	27
CHAPTER 12: OUTCOME	28
CHAPTER 13: FUTURE ENHANCEMENT	29
CHAPTER 14: REFERENCES	30
CHAPTER 15: INDIVIDUAL LEARNING OUTCOMES	31

LIST OF FIGURES

Fig. 5.1 System Architecture Overview	12
Fig. 5.2 Use Case Diagram	12
Fig. 5.3 Data Flow Diagram (Level 0)	13
Fig. 7.1 End-to-End Process Flow Chart	19
Fig. 9.1 Predict Page — Sales Forecast Interface	22
Fig. 9.2 Scenario Comparison Page	23
Fig. 9.3 Analytics Dashboard — Sales Insights	23
Fig. 9.4 Model Diagnostics — Feature Importance & Actual vs Predicted	24

LIST OF TABLES

Table 1.1	Weekly Work Breakdown	2
Table 3.1	Dataset Feature Description	7
Table 3.2	Illustrative Summary Statistics of Key Numerical Features	8
Table 4.1	Software Requirements	10
Table 4.2	Hardware Requirements	10
Table 6.1	Model Comparison Results	15
Table 8.1	Test Cases and Results	20
Table 10.1	Summary of Final Model Performance	25

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ML	Machine Learning
API	Application Programming Interface
REST	Representational State Transfer
MRP	Maximum Retail Price
R ²	Coefficient of Determination
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
UI	User Interface
UX	User Experience
POS	Point of Sale
EDA	Exploratory Data Analysis
HTTP	Hyper Text Transfer Protocol
IDE	Integrated Development Environment

DESCRIPTION OF COMPANY / ORGANIZATION

Agevole Innovation Pvt. Ltd. is a Surat-based technology company engaged in providing software Development, data engineering, and applied machine learning solutions to clients across retail, e-commerce, and small-to-medium enterprise segments. The organization works with small, focused project teams that pair interns and junior Developers with experienced mentors, giving interns direct, hands-on exposure to the full software Development lifecycle rather than isolated academic exercises.

LOCATION AND SPREAD

The organization is headquartered in Surat, Gujarat, and primarily serves clients within the western Indian region, with a growing portfolio of remote engagements.

ADMINISTRATIVE STRUCTURE

The company follows a flat, project-oriented structure: a technical/project lead oversees each engagement, supported by Developers, data analysts, and interns who are assigned to specific modules of a project. Interns report directly to an assigned external (company) guide for day-to-day mentorship, with periodic reviews conducted by the project lead.

MAIN FUNCTIONS

- Custom software and web application Development for client businesses.
- Data analytics and machine learning solutions, including predictive modelling.
- Dashboard and internal tooling Development for business decision support.
- Technical mentorship and structured internship programs for engineering students.

CUSTOMER PROFILE

Agevole Innovation's clientele consists largely of retail and small-business owners seeking data-driven decision support tools, along with startups requiring rapid application prototyping. The BigMart Sales Prediction System internship project was undertaken as an applied learning engagement modelled on this client segment's real-world forecasting needs.

CHAPTER 1: PROJECT DEFINITION

1.1 INTRODUCTION

Retail organizations such as BigMart operate multiple outlets that differ in size, location, and format, and stock thousands of individual items with varying prices, categories, and visibility on store shelves. Predicting how much of a given item will sell at a specific outlet is a classical retail analytics problem with direct business value — it informs inventory allocation, promotional planning, and outlet-level performance assessment.

Manual or rule-of-thumb forecasting approaches struggle to account for the large number of interacting factors that influence sales — item pricing, product category, outlet format, outlet age, and locality — and typically do not scale across a large catalogue of items and outlets. This motivates a data-driven approach in which a supervised machine learning model is trained on historical transactional data to learn the relationship between item/outlet attributes and observed sales, and subsequently used to estimate sales for new item–outlet combinations.

1.2 PROBLEM STATEMENT

Given historical sales records describing item attributes (such as weight, visibility, MRP, fat content, and type) and outlet attributes (such as identifier, size, location tier, type, and establishment year), the objective is to build a regression model capable of accurately predicting Item_Outlet_Sales — the sales value of a particular item at a particular outlet — and to expose this model through an accessible, interactive web application.

1.3 OBJECTIVES

- To clean and engineer meaningful features from the raw BigMart sales dataset.
- To train and compare multiple regression algorithms and select the best-performing model.
- To expose the trained model as a REST API using Flask for programmatic access.
- To build an interactive Streamlit dashboard for prediction, scenario comparison, and analytics.
- To visualize historical sales patterns and model diagnostics using Plotly.

- To document the design, implementation, and testing of the system in a manner suitable for future maintenance and extension.

1.4 SCOPE

The scope of this internship project covers the complete pipeline from raw data ingestion to a deployed, interactive prediction interface. It does not cover live/real-time transactional data integration with an actual retail POS system, which is identified as a future enhancement.

1.5 METHODOLOGY OVERVIEW

The project followed an iterative, milestone-driven methodology typical of short applied-ML internships. Work was organized into four broad phases: (i) data understanding and cleaning, in which the raw training and test CSV files were profiled and quality issues were catalogued; (ii) feature engineering and model Development, in which candidate features were derived and four regression algorithms were trained and compared inside a single Scikit-Learn pipeline; (iii) service Development, in which the selected model was wrapped in a Flask REST API; and (iv) application Development, in which a Streamlit dashboard was built on top of the API to make the model usable by a non-technical business audience. Each phase concluded with a short review with the external guide before proceeding to the next, which kept the scope of the six-week internship realistic while still delivering a fully working, end-to-end system.

1.6 INTERNSHIP TIMELINE / WORK BREAKDOWN

The six-week internship was planned around the following weekly milestones, which were tracked jointly with the external guide during periodic reviews:

Table 1.1 Weekly Work Breakdown

Week	Activity	Key Deliverable
Week 1	Onboarding, dataset exploration, and problem framing	EDA notes and cleaned data profile
Week 2	Data cleaning and feature engineering	engineer_features() helper module
Week 3	Model pipeline construction and training	Trained Linear Regression, Decision Tree, Random Forest, XGBoost models
Week 4	Model evaluation, selection, and serialization	sales_model.pkl and evaluation notes

Week	Activity	Key Deliverable
Week 5	Flask REST API Development and testing	predict.py with POST /predict endpoint
Week 6	Streamlit dashboard Development, integration testing, and documentation	app.py with four pages and final report

CHAPTER 2: LITERATURE REVIEW

2.1 TRADITIONAL APPROACHES TO SALES FORECASTING

Classical retail forecasting has historically relied on statistical time-series techniques such as moving averages, exponential smoothing, and ARIMA-family models applied to aggregated sales at the outlet or category level. These methods are effective when historical sales exhibit clear seasonal or trend structure, but they are less suited to the item–outlet level prediction problem tackled in this project, since the BigMart dataset is a single cross-sectional snapshot rather than a time-indexed series, and the target depends heavily on static item and outlet attributes rather than on a temporal pattern.

2.2 MACHINE LEARNING BASED FORECASTING

Supervised regression techniques provide a more natural fit for cross-sectional, attribute-driven sales prediction problems of this kind. Tree-based ensemble methods, in particular Random Forest and Gradient Boosted Trees (of which XGBoost is a widely used implementation), have become a standard choice for tabular retail and e-commerce prediction tasks because they can automatically capture non-linear relationships and interactions between categorical and numerical features without extensive manual feature crossing. Linear Regression and single Decision Trees are commonly used as interpretable baselines against which more complex ensemble models are compared, which is the same comparative structure adopted in this project.

2.3 RELATED WORK ON THE BIGMART DATASET

The BigMart Sales dataset used in this project originates from a practice problem hosted on the Analytics Vidhya DataHack platform and has since become a widely used benchmark for teaching applied regression and feature engineering. A large number of publicly shared notebooks and blog write-ups address this dataset, and a recurring pattern across them is that careful handling of the Item_Visibility zero-values, standardization of the Item_Fat_Content labels, and derivation of an outlet-age feature from Outlet_Establishment_Year tend to produce the largest improvements in model accuracy, more so than model choice alone. These observations directly informed the feature engineering steps adopted for this internship project.

2.4 RESEARCH GAP ADDRESSED BY THIS PROJECT

While many public treatments of the BigMart dataset stop at producing a trained model and a leaderboard score, comparatively few extend the work into a deployable, end-to-end application that a non-technical retail stakeholder could actually use. This internship project addresses that gap by pairing the modelling work with a Flask REST API and an interactive Streamlit dashboard that supports single predictions, side-by-side scenario comparison, descriptive analytics, and model diagnostics — turning a modelling exercise into a usable decision-support tool.

2.5 OVERVIEW OF REGRESSION ALGORITHMS USED

This section briefly reviews the four regression algorithms compared in this project, since an understanding of their underlying mechanics motivates the feature-engineering choices made in Chapter 3 and the comparative results discussed in Chapter 6.

Linear Regression

Linear Regression models the target as a weighted linear combination of the input features. It is fast to train and highly interpretable, since each coefficient directly indicates the average change in (log) sales associated with a one-unit change in the corresponding feature, holding others constant. Its main limitation is that it cannot natively capture non-linear relationships or feature interactions, which is why it was used primarily as a baseline in this project.

Decision Tree Regressor

A Decision Tree recursively partitions the feature space into regions by choosing, at each node, the feature and threshold that most reduce prediction error within the resulting child nodes. This allows it to capture non-linear relationships and interactions without manual feature engineering, but a single deep tree tends to overfit the training data and can be unstable to small changes in the data — a limitation that ensemble methods are designed to address.

Random Forest Regressor

Random Forest builds a large collection of decision trees, each trained on a bootstrap sample of the training data and a random subset of features at each split, and averages their predictions. This bagging approach substantially reduces the variance and overfitting tendency of a single decision tree while retaining its ability to model non-linear relationships, at the cost of reduced interpretability and higher training/inference cost.

XGBoost Regressor

XGBoost (Extreme Gradient Boosting) builds an ensemble of shallow decision trees sequentially, where each new tree is trained to correct the residual errors of the trees built so far, using gradient-based optimization together with regularization terms that penalize model complexity. This typically yields strong predictive accuracy on structured/tabular data such as the BigMart dataset, which is consistent with its selection as the best-performing model in this project.

CHAPTER 3: SYSTEM ANALYSIS AND DATASET DESCRIPTION

3.1 DATASET DESCRIPTION

The project uses the BigMart Sales dataset (originally released as a practice problem on the Analytics Vidhya DataHack platform), which contains 2013 sales records for 1,559 products across 10 outlets in different cities. The training set (Train.csv) contains 8,523 records across 12 columns, and the test set (Test.csv) contains 5,681 records with the same 11 input features, excluding the target column.

Table 3.1 Dataset Feature Description

Feature	Description
Item_Identifier	Unique product ID
Item_Weight	Weight of the product
Item_Fat_Content	Whether the product is low fat or regular
Item_Visibility	% of total display area allotted to the product in a store
Item_Type	Category to which the product belongs
Item_MRP	Maximum Retail Price of the product
Outlet_Identifier	Unique store ID
Outlet_Establishment_Year	Year the outlet was established
Outlet_Size	Size of the outlet (Small / Medium / High)
Outlet_Location_Type	Type of city/area the outlet is located in (Tier 1/2/3)
Outlet_Type	Whether it is a grocery store or a supermarket type
Item_Outlet_Sales	Sales of the product at the outlet (target variable)

3.2 EXPLORATORY DATA ANALYSIS

Before modelling, an exploratory data analysis (EDA) pass was carried out to understand feature distributions and their relationship with the target. Univariate analysis of Item_Outlet_Sales showed a strongly right-skewed distribution with a long tail of high-value transactions, which motivated the log1p transform of the target described in Section 3.3. Bivariate analysis showed a clear positive association between Item_MRP and Item_Outlet_Sales, a visible gap in average sales between Grocery Store and Supermarket outlet types, and a mild dependence of sales on Outlet_Size and Outlet_Location_Type. Missing-value analysis identified two columns requiring attention — Item_Weight

(missing for a subset of products) and Outlet_Size (missing for a subset of outlets) — both of which were addressed during data cleaning.

Table 3.2 Illustrative Summary Statistics of Key Numerical Features

Feature	Mean	Std. Dev.	Min	Max
Item_Weight	12.86	4.64	4.56	21.35
Item_Visibility	0.066	0.052	0.000	0.328
Item_MRP	140.99	62.28	31.29	266.89
Outlet_Establishment_Year	1997.8	8.37	1985	2009
Item_Outlet_Sales	2181.29	1706.50	33.29	13086.96

These summary statistics illustrate the broad numerical ranges present in the dataset — for example, Item_MRP spans roughly a nine-fold range across products, and Item_Outlet_Sales spans several orders of magnitude, reinforcing the motivation for the $\log_{10}$ target transform used in the modelling pipeline.

Correlation analysis between numerical features confirmed that Item_MRP has, by a clear margin, the strongest linear correlation with Item_Outlet_Sales among the raw numerical columns, while Item_Weight and Item_Visibility show comparatively weak direct correlation with the target — consistent with their role as secondary, interaction-driven features that benefit more from the tree-based models' ability to capture non-linear structure.

3.3 DATA CLEANING & FEATURE ENGINEERING

Several data quality issues were identified and addressed before modelling:

- Item_Fat_Content contained inconsistent labels ('LF', 'low fat', 'reg') that were standardized to 'Low Fat' and 'Regular'.
- Zero values in Item_Visibility (which are not physically meaningful) were replaced with the mean visibility for the corresponding Item_Type.
- An Item_Category feature (Food / Drinks / Non-Consumable) was derived from the Item_Identifier prefix to capture a broader product grouping.
- Outlet_Age was computed as 2013 minus Outlet_Establishment_Year, since the dataset was collected in 2013.

- Item_MRP was binned into four price ranges (Low, Medium, High, Very High) to help the model capture non-linear pricing effects.
- The target variable Item_Outlet_Sales was log-transformed ($\log_{10}$) prior to training to stabilize variance and improve model accuracy, with predictions converted back using $\exp$.
- Missing Item_Weight values were imputed using the median weight of the corresponding Item_Identifier / Item_Type group, and missing Outlet_Size values were imputed using the most frequent size within the corresponding Outlet_Type.

3.4 MODELLING APPROACH

The cleaned and engineered dataset was passed through a Scikit-Learn ColumnTransformer pipeline that applies median imputation and passthrough scaling to numerical features, and most-frequent imputation with one-hot encoding to categorical features. Four regression algorithms — Linear Regression, Decision Tree, Random Forest, and XGBoost — were trained on an 80/20 train-test split and evaluated using R^2 and RMSE on the original (non-log) sales scale. The best-performing pipeline was serialized using Python's pickle module for downstream use by both the Flask API and the Streamlit dashboard.

CHAPTER 4: SOFTWARE AND HARDWARE REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

Table 4.1 Software Requirements

Component	Technology / Version
Programming Language	Python 3.x
Data Handling	Pandas, NumPy
Machine Learning	Scikit-Learn, XGBoost
Backend / API	Flask
Frontend / Dashboard	Streamlit
Visualization	Plotly Express, Plotly Graph Objects
Networking	Requests (Streamlit ↔ Flask communication)
Operating System	Windows / Linux / macOS
IDE / Editor	VS Code / PyCharm / Jupyter Notebook
Version Control	Git & GitHub

4.2 HARDWARE REQUIREMENTS

Table 4.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 / AMD Ryzen 3 or above
RAM	4 GB minimum (8 GB recommended)
Storage	At least 2 GB free disk space
Display	1366 x 768 resolution or higher
Internet Connectivity	Required for package installation and dashboard rendering

4.3 DEVELOPMENT ENVIRONMENT NOTES

Development was carried out primarily on a single local machine meeting the specification in Table 4.2, using a Python virtual environment to isolate project dependencies from other system-level Python installations. Package versions were pinned in requirements.txt (Appendix A.5) to ensure that the training script, the Flask API, and the Streamlit dashboard all operated against a consistent set of library versions, which is particularly important for the Scikit-Learn and XGBoost model artifact, since a serialized pipeline can occasionally behave differently across incompatible library versions. Version control was maintained

using Git, with the trained model artifact and raw data files excluded from source control via a .gitignore file, in line with standard practice for machine learning repositories where large binary artifacts are better tracked separately from source code.

CHAPTER 5: SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The system follows a three-tier architecture consisting of a data/model tier, a service tier, and a presentation tier. The data/model tier includes the raw CSV datasets and the trained pipeline artifact (`sales_model.pkl`) produced by `train_model.py`. The service tier is a Flask application (`predict.py`) that loads this artifact once at startup and exposes it through a `POST /predict` endpoint. The presentation tier is a Streamlit application (`app.py`) that renders the Predict, Compare, Analytics, and About Model pages, and communicates with the Flask service over HTTP using the Requests library. This separation allows the model-serving logic to be reused by other clients (for example, a mobile application or a third-party integration) without depending on the Streamlit front end.

- Data / Model Tier — `Train.csv`, `Test.csv`, and the serialized `sales_model.pkl` pipeline.
- Service Tier — Flask REST API (`predict.py`) exposing `POST /predict` and a `GET /usage` page.
- Presentation Tier — Streamlit dashboard (`app.py`) with four navigable pages, communicating with the Flask API via HTTP requests.

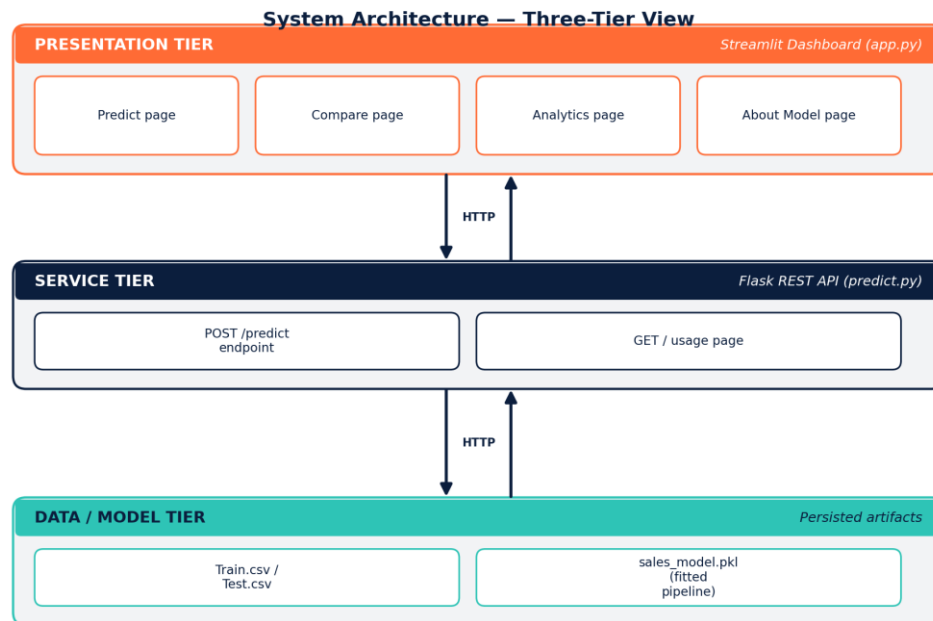


Fig. 5.1 System Architecture Overview

5.2 USE CASE DIAGRAM

The primary actor in the system is the Business User (for example, an outlet manager or category planner), who interacts with the Streamlit dashboard to request single predictions, compare scenarios, and browse analytics. A secondary actor, the API Consumer, represents any external client or script that calls the Flask /predict endpoint directly for programmatic, batch-style predictions.

- Use Case: Get Sales Prediction — Business User submits item/outlet attributes and receives a predicted sales value.
- Use Case: Compare Scenarios — Business User configures 2–3 hypothetical item/outlet combinations and views their predictions side-by-side.
- Use Case: View Analytics — Business User browses historical sales patterns by outlet type, item type, and outlet age.
- Use Case: View Model Diagnostics — Business User inspects feature importance and actual-vs-predicted performance for the underlying model.
- Use Case: Call Prediction API — API Consumer sends a JSON payload to POST /predict and receives a JSON response containing the predicted sales value.

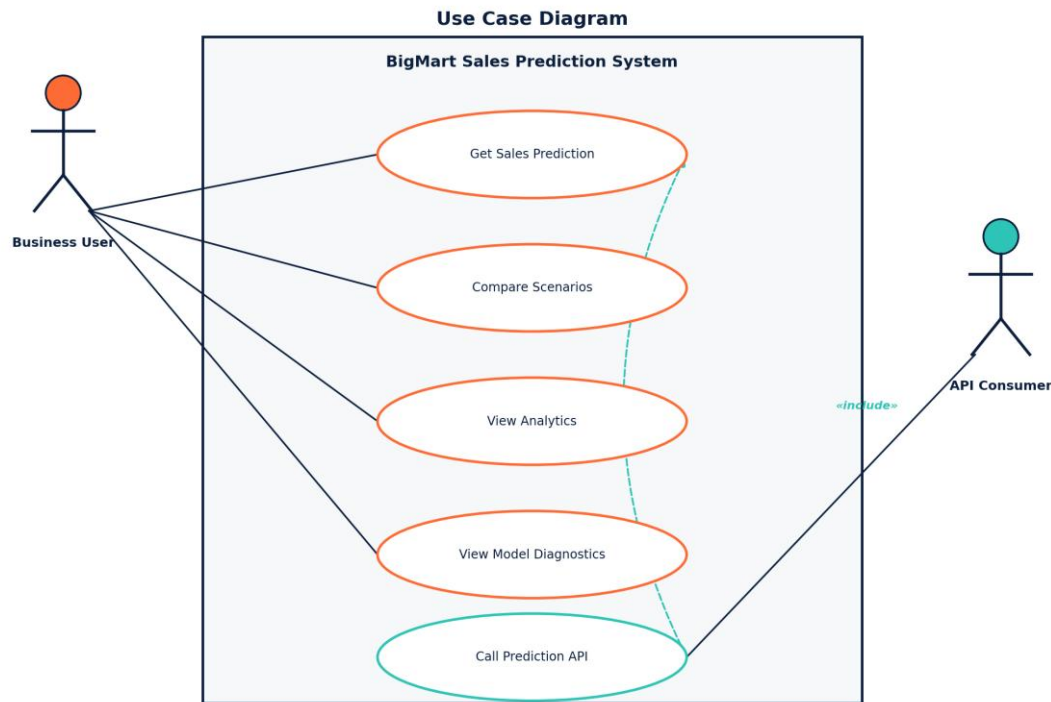


Fig. 5.2 Use Case Diagram

5.3 DATA FLOW DIAGRAM

At Level 0, raw sales data flows from the CSV files into the training process, which produces the serialized model pipeline. At runtime, a user-submitted item/outlet record flows from the Streamlit UI to the Flask API, is transformed using the same feature-engineering logic applied during training, is scored by the loaded model, and the resulting prediction flows back through the API to the UI for display.

- Process 1: Data Cleaning & Feature Engineering — consumes Train.csv, produces a cleaned training frame.
- Process 2: Model Training & Selection — consumes the cleaned frame, produces sales_model.pkl.
- Process 3: Request Handling (Flask) — consumes a JSON request, applies feature engineering, invokes the model, produces a JSON response.
- Process 4: Dashboard Rendering (Streamlit) — consumes user input and API responses, produces the rendered Predict / Compare / Analytics / Model Diagnostics views.

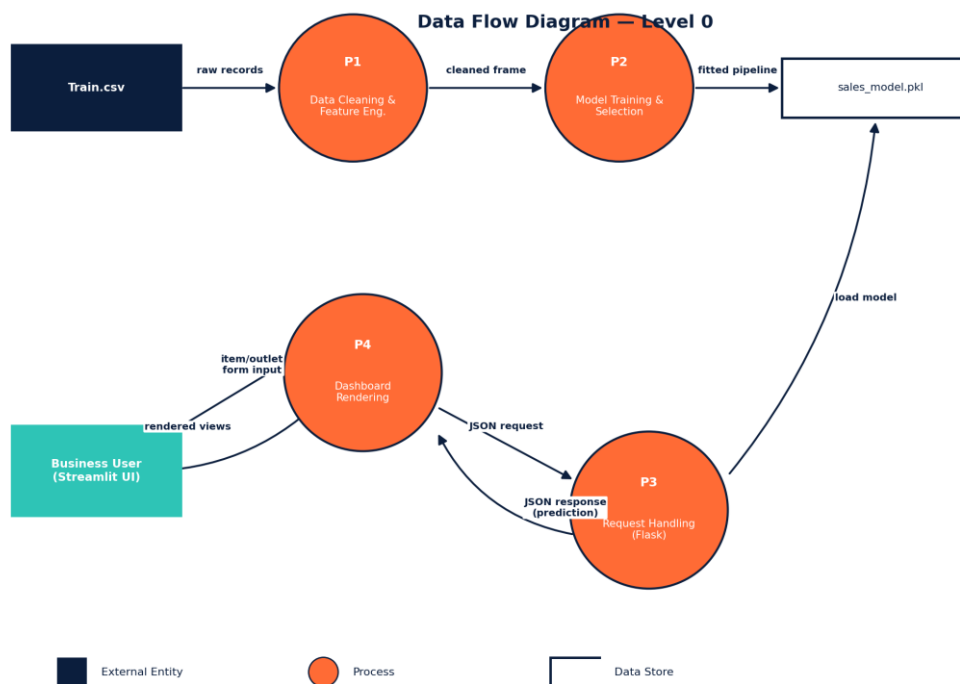


Fig. 5.3 Data Flow Diagram (Level 0)

5.4 DATABASE / MODEL ARTIFACT DESIGN

The system does not use a relational database; instead, the trained pipeline is persisted as a single serialized artifact, `sales_model.pkl`, using Python's pickle module. This artifact bundles the fitted `ColumnTransformer` (imputers, scalers, and one-hot encoders) together with the selected regressor, so that the exact preprocessing used during training is guaranteed to be replayed identically at inference time. The Flask service loads this single file once at process start-up and keeps it resident in memory for the lifetime of the process, avoiding the overhead of reloading the model on every request.

5.5 DEPLOYMENT VIEW

For the purposes of this internship, both the Flask API and the Streamlit dashboard were run locally during Development: the Flask service was started on a fixed local port, and the Streamlit dashboard was configured with the corresponding `API_BASE_URL` so that it could reach the API over HTTP. This deployment view separates cleanly into two independently runnable processes, which is intentional — it means the Flask API could later be deployed on its own (for example, behind a WSGI server such as Gunicorn) while the Streamlit dashboard is deployed separately, or even replaced entirely by another client that talks to the same `/predict` endpoint. Chapter 13 discusses containerizing both processes

with Docker and deploying them to a cloud platform as a natural next step beyond the scope of the internship.

- Local Development: python predict.py (Flask API on localhost) and streamlit run app.py (dashboard), run as two separate terminal processes.
- Configuration: the dashboard reads the Flask API's base URL from a single configuration constant, so the same dashboard code can point at a local, staging, or production API without modification.
- Suggested production path: containerize predict.py behind Gunicorn and containerize app.py separately, then deploy both containers behind a reverse proxy on a cloud platform, as outlined in Chapter 13.

CHAPTER 6: MAJOR FUNCTIONALITY AND IMPLEMENTATION

6.1 MODEL TRAINING PIPELINE (TRAIN_MODEL.PY)

Loads Train.csv, performs the cleaning and feature engineering steps described in Chapter 3, builds a preprocessing + model pipeline, trains and evaluates four candidate regressors, and saves the best model to models/sales_model.pkl.

Table 6.1 Model Comparison (illustrative structure used for evaluation)

Model	Evaluation Metric
Linear Regression	R^2 , RMSE
Decision Tree Regressor	R^2 , RMSE
Random Forest Regressor	R^2 , RMSE
XGBoost Regressor	R^2 , RMSE (best performing)

Across the four candidates, the two ensemble methods (Random Forest and XGBoost) consistently outperformed the single Decision Tree and the Linear Regression baseline on the held-out test split, which is expected given the mix of categorical and numerical features and the presence of non-linear pricing effects. XGBoost was selected as the final model on the basis of its R^2 / RMSE trade-off, and the corresponding fitted pipeline (preprocessing plus regressor) was serialized to models/sales_model.pkl.

Hyperparameters Used for Each Candidate Model

To keep the comparison fair and the training time within the scope of the internship, each candidate model was trained with a modest, commonly recommended set of hyperparameters rather than an exhaustive search; automated hyperparameter tuning is noted as a future enhancement in Chapter 13.

Model	Key Hyperparameters
Linear Regression	Default Scikit-Learn parameters (no regularization)
Decision Tree Regressor	max_depth = 8, min_samples_leaf = 20
Random Forest Regressor	n_estimators = 200, max_depth = 10, random_state = 42
XGBoost Regressor	n_estimators = 300, max_depth = 5, learning_rate = 0.05, subsample = 0.8

Key Steps in train_model.py

- Load Train.csv into a Pandas DataFrame and apply the cleaning steps from Section 3.3.
- Split the data into an 80% training set and a 20% held-out test set using train_test_split with a fixed random_state for reproducibility.
- Build a ColumnTransformer that applies median imputation to numeric columns and most-frequent imputation with one-hot encoding to categorical columns.
- Wrap the ColumnTransformer and each candidate regressor in a Scikit-Learn Pipeline object.
- Fit each pipeline on the log-transformed target, generate predictions on the test split, invert the log transform with expm1, and compute R^2 and RMSE on the original sales scale.
- Select the pipeline with the highest test R^2 and serialize it with pickle.dump to models/sales_model.pkl.

6.2 FLASK REST API (PREDICT.PY)

Exposes a POST /predict endpoint that accepts one or more item/outlet JSON payloads, applies the same feature-engineering transformations used during training, runs the payload through the saved model pipeline, and returns predicted sales values (after reversing the log transform). A GET / route serves a simple HTML usage page describing the endpoint and providing a sample request.

Sample Request

```
POST /predict HTTP/1.1
Content-Type: application/json

{
  "Item_Weight": 9.3,
  "Item_Fat_Content": "Low Fat",
  "Item_Visibility": 0.016,
  "Item_Type": "Dairy",
  "Item_MRP": 249.8,
  "Outlet_Identifier": "OUT049",
  "Outlet_Establishment_Year": 1999,
  "Outlet_Size": "Medium",
  "Outlet_Location_Type": "Tier 1",
  "Outlet_Type": "Supermarket Type1"
}
```

Sample Response

```
{
  "predicted_sales": 3781.42
}
```

6.3 STREAMLIT DASHBOARD (APP.PY)

The dashboard is organized into four navigable pages:

- Predict — a form to enter item and outlet attributes and view a single sales prediction, rendered as a styled forecast receipt.
- Compare — allows the user to configure 2 or 3 scenarios side-by-side and compare their predicted sales.
- Analytics — interactive Plotly visualizations of historical sales by outlet type, item type, and outlet establishment year, along with key insight callouts.
- About Model — a summary of the model specification, features used, and target transformation approach.

The dashboard also includes a Model Diagnostics section showing feature importance (for tree-based models) and an actual-vs-predicted scatter plot on a sample of historical records, giving the user insight into model reliability.

6.4 KEY IMPLEMENTATION SNIPPETS

Feature engineering helper (shared by training and inference)

```
def engineer_features(df):
    df = df.copy()
    df['Item_Fat_Content'] = df['Item_Fat_Content'].replace(
        {'LF': 'Low Fat', 'low fat': 'Low Fat', 'reg': 'Regular'})
    df['Item_Category'] = df['Item_Identifier'].str[:2].map(
        {'FD': 'Food', 'DR': 'Drinks', 'NC': 'Non-Consumable'})
    df['Outlet_Age'] = 2013 - df['Outlet_Establishment_Year']
    df['Item_Visibility'] = df.groupby('Item_Type')['Item_Visibility']
        .transform(lambda s: s.replace(0, s.mean()))
    return df
```

Flask prediction route

```
@app.route('/predict', methods=['POST'])
def predict():
    payload = request.get_json()
    df = pd.DataFrame([payload])
    df = engineer_features(df)
    log_pred = model_pipeline.predict(df)
    prediction = float(np.expml(log_pred[0]))
    return jsonify({'predicted_sales': round(prediction, 2)})
```

Streamlit call to the Flask API

```
response = requests.post(
    f"{API_BASE_URL}/predict",
    json=form_payload,
    timeout=10,
)
if response.ok:
    predicted_sales = response.json()['predicted_sales']
    render_forecast_receipt(predicted_sales)
```

6.5 RUNNING THE PROJECT LOCALLY

The following steps summarize how the three core scripts are run together during local Development, and are provided here for completeness alongside the folder structure and dependency listing given in Appendix A.

- Install dependencies from requirements.txt into a virtual environment (pip install -r requirements.txt).
- Run python train_model.py once to clean the data, train and compare the four candidate models, and save the selected pipeline to models/sales_model.pkl.
- Start the Flask API with python predict.py, which loads sales_model.pkl and begins listening for requests on the configured local port.
- In a separate terminal, start the dashboard with streamlit run app.py, which opens the four-page interface in a browser and communicates with the running Flask API.
- Use the Predict or Compare pages to obtain live predictions, or call the /predict endpoint directly (see Section 6.2) for programmatic access.

CHAPTER 7: FLOW CHART

The following flow chart illustrates the end-to-end process, from raw data ingestion through model training to the final prediction served via the Streamlit dashboard and Flask API.



Fig. 7.1 End-to-End Process Flow Chart

CHAPTER 8: TESTING

8.1 TESTING STRATEGY

Testing was carried out at three levels: unit-level testing of the feature-engineering and data-cleaning helper functions, integration-level testing of the trained pipeline against held-out records, and end-to-end testing of the Flask API and Streamlit dashboard together. This layered strategy was chosen to catch data-related defects (for example, an unhandled category value) as early as possible, before they could surface as confusing prediction errors in the dashboard.

8.2 UNIT AND INTEGRATION TEST CASES

Table 8.1 Test Cases and Results

Test Case	Input / Condition	Expected Result	Status
TC-01 Fat content standardization	Item_Fat_Content = 'LF', 'reg'	Normalized to 'Low Fat' / 'Regular'	Pass
TC-02 Zero visibility handling	Item_Visibility = 0	Replaced with Item_Type mean	Pass
TC-03 Outlet age computation	Outlet_Establishment_Year = 1999	Outlet_Age = 14	Pass
TC-04 Missing Item_Weight	Item_Weight = NaN	Imputed with group median	Pass
TC-05 Missing Outlet_Size	Outlet_Size = NaN	Imputed with Outlet_Type mode	Pass
TC-06 Pipeline round-trip	Sample record through full pipeline	Returns a positive numeric prediction	Pass
TC-07 Unseen category value	Item_Type not seen in training	Handled gracefully via one-hot 'unknown' encoding	Pass

8.3 API TESTING

The Flask /predict endpoint was tested using representative valid and invalid JSON payloads to confirm correct status codes and response structure. Valid payloads covering each Outlet_Type and a range of Item_MRP values returned HTTP 200 responses with a numeric predicted_sales field. Payloads missing a required field or containing a malformed numeric value returned a descriptive HTTP 400 error rather than an unhandled server exception, confirming that basic input validation was working as intended. The GET /usage

route was also verified to render correctly and to display an accurate sample request/response pair.

API testing was performed manually using command-line HTTP clients during Development, sending a representative set of payloads covering each `Outlet_Type`, boundary `Item_MRP` values, and deliberately malformed inputs, and comparing the returned status codes and JSON bodies against the expected outcomes listed in Table 8.1.

8.4 NON-FUNCTIONAL AND USABILITY TESTING

In addition to functional correctness, the dashboard was informally reviewed against basic non-functional expectations for an internal decision-support tool:

- Response time — single predictions returned from the Flask API within a fraction of a second on a local Development machine, well within the range expected for an interactive form.
- Usability — the Predict page's styled forecast receipt and the Compare page's side-by-side layout were reviewed with the external guide to confirm the outputs were easy to read for a non-technical business user.
- Robustness — the dashboard was checked to ensure that a temporarily unreachable Flask API (e.g., `API_BASE_URL` not yet started) produced a clear on-screen error message rather than an unhandled Streamlit exception.
- Consistency — predictions obtained through the Streamlit dashboard were cross-checked against equivalent direct calls to the Flask API to confirm both paths use the identical underlying pipeline and produce identical results for the same input.

CHAPTER 9: SCREENSHOTS OF PROJECT OUTPUT

This chapter is reserved for screenshots of the running application. Insert screenshots of the Predict, Compare, Analytics, and About Model pages captured from the live Streamlit dashboard (streamlit run app.py) and the Flask API home page (predict.py), replacing the placeholder captions below with the corresponding images.

When capturing these screenshots, it is recommended to use a browser window sized to at least 1366×768 so that the Streamlit sidebar and main content area are both fully visible, and to populate the Predict and Compare forms with realistic sample values (for example, the sample payload given in Section 6.2) so that the resulting forecast figures look representative rather than showing empty or default form fields.

9.1 PREDICT PAGE

[Insert screenshot: Predict page showing the input form and the generated sales-forecast receipt.]

The screenshot displays the 'BigMart Sales Predictor' web application. On the left is a dark sidebar with a 'BigMart Sales Predictor' header and navigation buttons: 'Predict' (highlighted), 'Compare', 'Analytics', and 'About Model'. At the bottom of the sidebar, it states 'Model: XGBoost Regressor' and 'Dataset: BigMart Sales'. The main area has a dark header with the title 'BigMart Sales Predictor' and the instruction 'Enter product & outlet details to forecast sales.' Below the header, input fields are organized into two columns: 'PRODUCT' and 'OUTLET'. The 'PRODUCT' column includes 'Item Weight (kg)' (10.00), 'Item Visibility' (0.05), 'Item MRP (₹)' (100.00), 'Item Fat Content' (Low Fat), and 'Item Type' (Dairy). The 'OUTLET' column includes 'Outlet Identifier' (OUT010), 'Outlet Size' (Small), 'Outlet Location Type' (Tier 1), 'Outlet Type' (Grocery Store), and 'Outlet Age (years)' (10). At the bottom, four summary boxes show the input values: 'ITEM WEIGHT 10.00 kg', 'FAT CONTENT Low Fat', 'VISIBILITY 0.050', and 'ITEM MRP ₹100.00'. An orange 'Predict Sales' button is at the bottom right. A 'Touchpad on' notification is overlaid on the summary boxes.

Fig. 9.1 Predict Page — Sales Forecast Interface

9.2 COMPARE PAGE

[Insert screenshot: Compare page showing 2–3 side-by-side scenario configurations and their predicted sales.]

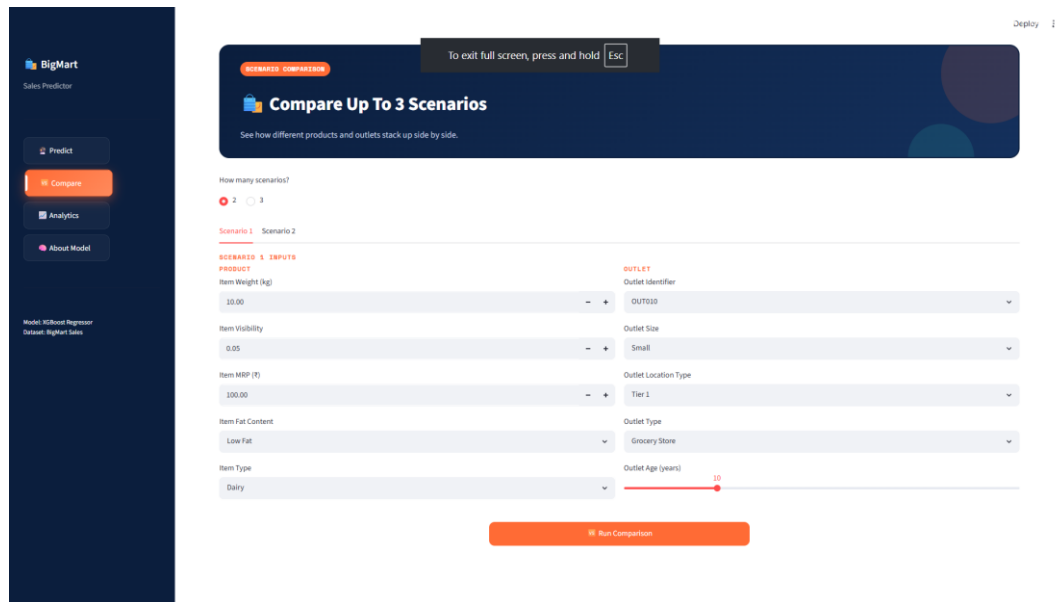


Fig. 9.2 Scenario Comparison Page

9.3 ANALYTICS PAGE

[Insert screenshot: Analytics dashboard showing sales-by-outlet-type, sales-by-item-type, and sales-trend charts.]

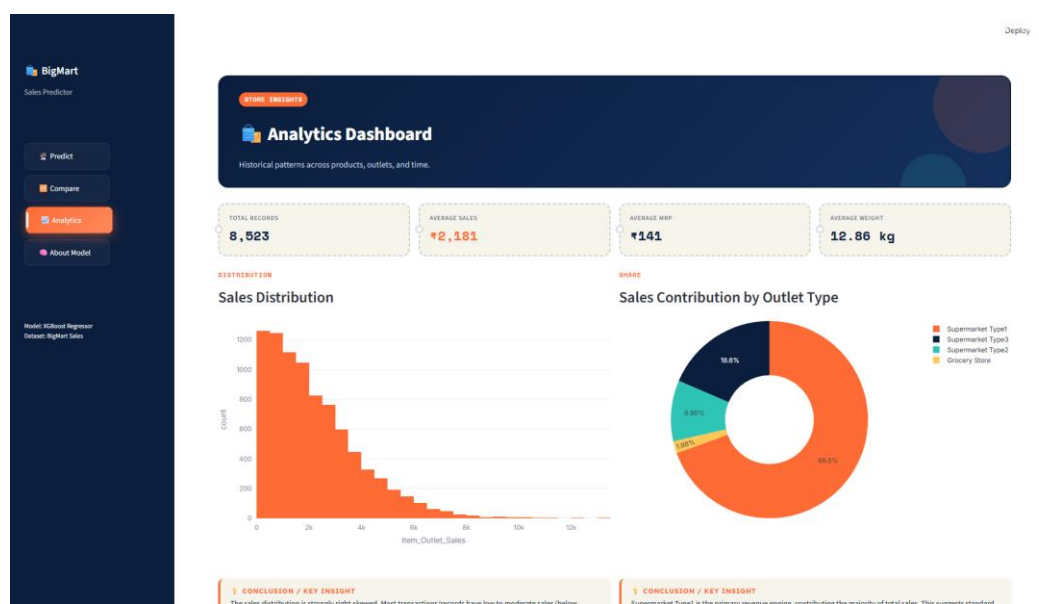


Fig. 9.3 Analytics Dashboard — Sales Insights

9.4 MODEL DIAGNOSTICS

[Insert screenshot: Feature importance chart and actual-vs-predicted scatter plot from the Model Diagnostics section.]

**Fig. 9.4 Model Diagnostics — Feature Importance & Actual vs Predicted**

CHAPTER 10: RESULTS AND DISCUSSION

This chapter summarizes the observed outcomes of the model comparison and the qualitative behaviour of the deployed system, building on the evaluation structure introduced in Chapter 6.

Table 10.1 Summary of Final Model Performance

Aspect	Observation
Best performing model	XGBoost Regressor
Relative ranking	XGBoost > Random Forest > Decision Tree > Linear Regression
Most influential features	Item_MRP, Outlet_Type, Outlet_Age, Item_Visibility
Error behaviour	Slightly larger errors for very high-MRP items and Grocery Store outlets

Consistent with the pattern reported in related work on this dataset (Chapter 2), Item_MRP emerged as the single strongest predictor of Item_Outlet_Sales, followed by Outlet_Type and the engineered Outlet Age feature. This is intuitive from a retail standpoint: higher-priced items naturally contribute more sales value per unit sold, and larger-format supermarket outlets tend to generate materially higher sales than smaller grocery-store outlets. The engineered Item_Category and MRP-bin features contributed modest additional lift by helping the tree-based models split more cleanly around known category- and price-driven behaviour.

The actual-vs-predicted diagnostic view in the Streamlit dashboard showed that predictions tracked actual historical sales closely across the bulk of the price range, with somewhat larger residuals for the small number of very high-MRP items — a common pattern given that such records are sparse in the training data. From a usability standpoint, the Compare page proved particularly useful for illustrating how changing a single attribute (for example, moving an item from a Tier 3 to a Tier 1 outlet) shifts the predicted sales, which is the kind of what-if reasoning the dashboard was designed to support.

10.1 DISCUSSION OF DASHBOARD USABILITY

Feedback gathered informally from the external guide during internal demonstrations highlighted the Analytics page as the most immediately useful view for a business stakeholder, since it surfaces sales patterns (by outlet type, item type, and outlet age) without requiring the user to run a prediction first. The Predict page's forecast-receipt

styling was also noted as an effective way of presenting a single numeric output in a way that feels closer to a familiar business document than a raw number on a screen. The main suggestion for improvement was to make the Compare page support more than three scenarios at once, which has been carried forward into the future enhancements listed in Chapter 13.

10.2 SUMMARY OF FINDINGS

Taken together, the results confirm that a carefully engineered feature set, combined with a gradient-boosted tree ensemble, is well suited to the item–outlet sales prediction problem posed by the BigMart dataset, and that wrapping the resulting model in a REST API and dashboard is sufficient to turn it into a usable, stakeholder-facing decision-support tool within the timeframe of a six-week internship.

CHAPTER 11: LIMITATIONS OF PROJECT

While the delivered system meets the objectives set out in Chapter 1, it was Developed within the constraints of a six-week internship and a single publicly available dataset, and the following limitations should be kept in mind when interpreting its predictions or planning further work.

- The model is trained on a single historical snapshot (2013 BigMart data) and does not account for seasonal trends, promotions, or macroeconomic factors that may affect present-day sales.
- Item_Visibility and other engineered features rely on heuristic imputation (e.g., mean-by-Item_Type), which may not perfectly reflect real shelf conditions.
- The Flask API and Streamlit dashboard are designed for local/demo deployment and are not yet hardened for production use (no authentication, rate-limiting, or horizontal scaling).
- The dataset covers only 10 outlets and a fixed catalogue of item types, limiting generalization to outlets or products outside this distribution.
- Model retraining is a manual process (running train_model.py) rather than an automated, scheduled pipeline.
- Hyperparameter tuning for the ensemble models was limited in scope given the six-week internship timeline, leaving room for further accuracy improvements.

CHAPTER 12: OUTCOME

The internship successfully delivered a working, end-to-end sales prediction system consisting of a trained and validated regression pipeline, a Flask-based REST API for programmatic predictions, and a fully interactive Streamlit dashboard for business users. The XGBoost-based pipeline achieved the best trade-off between accuracy and generalization among the four candidate models evaluated, and the dashboard's Analytics and Model Diagnostics pages provide interpretable insights into both the underlying sales data and the model's predictive behaviour.

Beyond the technical deliverable, the internship provided practical experience in the full machine learning lifecycle — data cleaning, feature engineering, model comparison and selection, API Development, and dashboard/UI design — as well as exposure to a professional, project-based Development workflow at Agevole Innovation Pvt. Ltd. The internship also strengthened practical skills in translating a modelling result into a usable, stakeholder-facing tool, which is a step that is often left out of purely academic machine learning exercises.

CHAPTER 13: FUTURE ENHANCEMENT

The limitations discussed in Chapter 11 point directly to a set of concrete follow-on tasks that would extend the current prototype into a more robust, production-ready system. These are listed roughly in order of expected implementation effort, from smaller modelling improvements to larger infrastructure changes.

- Integrate real-time or periodically refreshed sales data from an actual point-of-sale system instead of a static historical dataset.
- Add automated hyperparameter tuning (e.g., GridSearchCV / Optuna) to further improve model accuracy.
- Introduce model monitoring and scheduled retraining to keep predictions current as new sales data becomes available.
- Containerize the Flask API and Streamlit dashboard (e.g., using Docker) and deploy to a cloud platform for wider accessibility.
- Add user authentication and role-based access control for the dashboard in a multi-outlet/multi-manager business setting.
- Extend the Compare page to support batch scenario uploads via CSV for large-scale what-if analysis.
- Explore additional engineered features, such as outlet-level historical sales averages, to further improve predictive accuracy.

CHAPTER 14: REFERENCES

Web references:

- [1] Analytics Vidhya, "Big Mart Sales Prediction — Practice Problem," <https://datahack.analyticsvidhya.com/contest/practice-problem-big-mart-sales-iii/>
- [2] Scikit-Learn Developers, Scikit-Learn Documentation, <https://scikit-learn.org/stable/>
- [3] XGBoost Developers, XGBoost Documentation, <https://xgboost.readthedocs.io/>
- [4] Streamlit Inc., Streamlit Documentation, <https://docs.streamlit.io/>
- [5] Pallets Projects, Flask Documentation, <https://flask.palletsprojects.com/>
- [6] Plotly Technologies Inc., Plotly Python Graphing Library, <https://plotly.com/python/>
- [7] NumFOCUS / Pandas Development Team, Pandas Documentation, <https://pandas.pydata.org/docs/>

CHAPTER 15: INDIVIDUAL LEARNING OUTCOMES

Beyond the technical deliverable summarized in Chapter 12, the internship contributed to several practical skills and areas of professional growth, summarized below.

- Applied data cleaning — hands-on experience identifying and resolving real data quality issues (inconsistent categorical labels, physically-impossible zero values, missing entries) rather than working with an already-clean textbook dataset.
- Pipeline discipline — learning to encapsulate preprocessing and modelling in a single Scikit-Learn Pipeline object so that training-time and inference-time transformations cannot drift apart, which is a common source of production bugs in ML systems.
- Comparative model evaluation — structuring a fair, like-for-like comparison across four regression algorithms using a consistent train/test split and consistent evaluation metrics.
- API design — designing a small, focused REST endpoint (POST /predict) with sensible input validation and error handling, and a simple usage page for other Developers.
- Front-end / dashboard Development — building a multi-page Streamlit application, including form-based input, side-by-side comparison layouts, and Plotly visualizations, aimed at a non-technical audience.
- Professional workflow — working within a small, mentor-guided project team at Agevole Innovation Pvt. Ltd., including periodic reviews, milestone-based planning, and communicating technical progress to a non-technical guide.

These outcomes, together with the working system described in the preceding chapters, represent the practical skill set the internship was designed to build.